Programming Assignment 1 2024

ICS 3201 Computer Security

Due **Tuesday Nov. 12, 11:59pm**

1 Introduction

In many software systems today, the primary weakness often lies in the user's password. This is especially apparent in light of recent security breaches that have highlighted some of the weak passwords people commonly use (e.g., `123456` or `password`). It is very important, then, that users choose strong passwords (or "passphrases") to secure their accounts, but strong passwords can be long and unwieldy. Even more problematic, the user generally has many different services that use password authentication, and therefore the user must recall many different passwords.

One way for users to address this problem is to use a password manager, such as BitWarden and 1Password. Password managers make it very convenient for users to use a unique, strong password for each service that requires password authentication. However, given the sensitivity of the data contained in the password manager, one must take considerable care to store the information securely.

In this assignment, you will be writing a secure and efficient password manager. In your implementation, you will make use of various cryptographic primitives we have discussed in class—notably, authenticated encryption and collision-resistant hash functions. Because it is ill-advised to imple- ment your own primitives in cryptography, you should use an established library: in this case, the SubtleCrypto. We will provide starter code that contains a basic template, which you will be able to fill in to satisfy the functionality and security properties described below.

***Caveat**: Please do not consider this project as a substitution for a safe password manager. There are more security considerations that we do not consider in this project to make this password manager truly secure.*

2 Secure Password Manager

2.1 Implementation

In general, a password manager (also called a *keychain*) application will store its password database on disk, protected by a strong master password. While it is in use, it may store an "unlocked" representation of the database in memory from which it can provide the password for each desired domain. Instead of implementing a full standalone password manager application, for this project you will only be responsible for the core library. Thus, you will not need to implement the interactive front-end for interacting with the password manager, nor will you need to actually write the contents to disk. Instead, you will simulate these functionalities by providing features to serialize and deserialize your data structures to string representations, so that it would be easy to complete a full password manager application by writing these representations to disk.

Your password manager will keep its in-memory password data in a key-value store (KVS), represented by a Javascript object whose keys correspond to domain names, and whose values correspond to passwords for that domain. For example, a sample password manager instance might store the following information:

Key	Value
www.google.com	password
www.example.com	123456
www.amazon.com	6U)qA1OBy%3SZX$o
www.ebay.com	guest

Naturally, writing this information to disk in the clear is not secure. In this assignment, you will need to preserve both the confidentiality and the integrity of the values in your KVS. Thus, you will be encrypting all the values (i.e, the passwords for different domains) using an authenticated encryption scheme, namely AES-GCM. In order to accommodate a potentially large number of entries in the password manager, you will encrypt and store each record individually in-memory. In other words, it is not appropriate to encrypt the entire KVS as a single blob. This way, you do not have to decrypt every entry in the password manager when fetching a single record. We will also be using HMAC as our pseudorandom function (PRF).

We do not want to leak any information about the domains the user has stored in the password manager. At the same time, we want to maintain the ability to search for the data corresponding to a specific domain. In this assignment, the KVS (Javascript object) storing the password data should have as its keys the HMAC of each domain name, rather than the actual domain name in the clear.[1] Then, to look up the data corresponding to domain name x, you first compute HMAC(k, x), where k is the secret key for HMAC, and check whether the result exists as a key in the key-value store.

If you simply encrypt each domain/password pair in the KVS directly, your implementation will probably leak some information about the lengths of the passwords. We will not consider such implementations secure; rather, your implementation must prevent an adversary from learning any information about the password lengths. (To make this feasible, you may assume that the maximum length of any password is 64 characters.)

You should use the password-based key derivation function, PBKDF2, to derive keys from a master password provided by the user. We will discuss the details of PBKDF2 later on in the course, but we explain how to use it in Section 2.3. PBKDF2 is deliberately designed to be slow, and therefore you want to call it just once to derive a master key k. Once you have the output k of PBKDF2, you can use it as a key for HMAC to derive the two sub-keys you need: one sub-key to MAC the domain names, and one sub-key for encrypting passwords using AES-GCM. To derive these two sub-keys you can evaluate HMAC at two different arbitrary values using k as the HMAC key. Here we are using HMAC as a secure PRF.

Note that a secure password manager is *not allowed* to include the master password (or even a hash of it or any other values that would leak information about it), or any of the secret keys used, in the serialized password database on disk. Additionally, you should assume that an adversary has access to your source code – this means you cannot embed any values you hope to hide from the adversary in your code.

[1] Technically, you will need to use some serialized string representation (like Base64) of the HMAC value.

As another layer of defense, you will be asked to produce a SHA-256 checksum of the entire contents of the database when it is serialized for disk storage. This will be used to defend against certain attacks described in the next section.

2.2 Threat model

When designing any system with security goals, it is important to specify a threat model: i.e., we must precisely define the power of the adversary, as well as the condition the adversary must satisfy in order to be considered to have "broken" the system. Thus, we will now specify the threat model for our secure password manager, in the form of a security game (of the same flavor as the PRF or CPA games). In particular, just as the CPA game allows the adversary to specify messages of its choice, our definition will seem to give the adversary a great deal of power over the contents ofthe password database. It is important to remember that we must make such strong assumptionswhen attempting to show that a system is secure for general use, because we have no idea under what circumstances it may end up being deployed.

Our security game proceeds as follows. As usual, the password manager will play the role of the challenger—interacting with another implicit party, the disk storage—while the adversary will make a series of adaptive queries that determine the behavior of the system. Some of the adversary's queries may include a contingency for each of two possible experiments—as in experiments 0 and 1 in the CPA security game—and, as in the CPA game, the "experiment bit" parameter, b, will determine which series of queries the challenger actually executes. Each query will take one of the following forms:

1. Specify values $\langle \text{domain}, \text{password}_0, \text{password}_1 \rangle$ to be added to the database. In experiment 0, the challenger must run the password manager algorithm to add the domain-password pair $\langle \text{domain}, \text{password}_0 \rangle$ to the database, while in experiment 1, the challenger must run the same algorithm, but on the pair $\langle \text{domain}, \text{password}_1 \rangle$.
2. Specify a key (domain) that the challenger must remove from the password database.
3. Specify that the challenger must serialize the database state to "disk", whereupon the adversary will receive the entire result of the serialization, and will be able to replace it with an alternative of its choice (which must then immediately be deserialized by the challenger, running the password manager algorithm).
4. Specify a key (domain) for authentication, at which point the challenger (again running the password manager algorithms) must send the adversary the password corresponding to that domain.[2]

As in the PRF and CPA security games, we say that a password manager is secure if all computationally efficient adversaries have only negligible advantage in the game described above. (i.e. the adversary's probability of outputting 1 as its guess of the experiment bit b differs only by a negligible amount when in experiment 0 and when in experiment 1.) Unlike the PRF and CPA games, however,[3] we will need an additional restriction for our security definition here. In particular, we will only allow adversaries whose queries are "admissible" in the following sense:

[2] This ability models the fact that in real life, the adversary may control a service for which the client has stored a password.

[3] But similar to the CCA game.

- Whenever the adversary makes a query (4), requiring the challenger to send its password for a given domain *d*, on its *last* query (1) setting the password for *d* (if there was such a query at all), it must have been the case that $\text{password}_0 = \text{password}_1$.

(Indeed, it is fairly easy to see that without this restriction, the adversary could trivially win the game—by query (1), it could cause the challenger to add a password for some domain *d*, under the adversary's control, so that the password value differed between experiments 0 and 1; and then require, by query (4), that the challenger authenticate to it using domain *d*'s password.)

Intuitively, this definition captures the fact that even if the adversary is able to exert substantial control over the contents of the password database—and even if it controls some malicious remote servers—it still cannot learn anything about the passwords in the database for any *other* servers.

For this project, you will not be required to give a formal proof that your system fulfills the strong security definition we have just stated. However, your system **should** be secure under this threat model (and a proof should exist, even though you do not have to produce it).

However, you should note that, to satisfy such a strong definition, there are a number of interesting attacks that you will have to defend against, most notably **swap attacks** and **rollback attacks**. In a swap attack, the adversary interchanges the values corresponding to different keys. For instance, the adversary might switch the entries for `www.google.com` and `www.evil.com`. Then, when the user (for whatever reason) tries to authenticate to `www.evil.com`, the user inadvertently provides its credentials for `www.google.com`. It should be easy to see that an adversary able to perform a swap attack can easily win the security game we outlined above. In your implementation, you must provide a defense against a swap attack.

In a rollback attack, the adversary can replace a record with a previous version of the record. For example, suppose the adversary was able to retrieve the KVS in the example above. At some later time, the user changes her/his password for `www.google.com` to `google_pwd`, which would update the value for `www.google.com` in the KVS. However, the adversary can replace this updated record with the previous record for `www.google.com`. Note that, as in the previous section, merely using authenticated encryption does not protect against this attack. Rather, in your implementation, you should compute a SHA-256 hash of the contents of the password manager. You can assume this hash value can be saved to a trusted storage medium (inaccessible to the adversary)—such as a flash drive on the user's person. Whenever you load the password manager from disk, you should verify that the hash is valid. This way, you can be assured that the contents of the KVS have not been tampered with.

Depending on your design, your defense against rollback attacks might also turn out to protect against the swap attacks described earlier. However, you **must still implement** an explicit defense against swap attacks. In other words, the defenses you develop must work **independently** of one another. Even if a SHA-256 hash is not provided from trusted storage, your scheme must be secure against an adversary that swaps two records.

2.3 Using PBKDF2

In the previous section, we did not give a precise formulation of the security properties we are assuming of PBKDF2. To give the full picture, we would need to work in a framework called the "random oracle model," which would take us too far afield. Instead, this section will provide some

practical guidelines on how to use PBKDF2.

Although we will discuss this in more detail towards the end of the course, it is important to remember that when deriving anything using passwords, we should always use a randomly generated salt. The recommended length of the salt is 128 bits. The key derived from PBKDF2 will be a function of the provided password and the salt. To generate salt for a new Keychain, we provide a function `getRandomBytes()`. If you wish to derive the same key again in the future, such as when you're loading the keychain from disk, you will need to store the salt in the clear.

The idea behind salting passwords is to prevent an *offline dictionary attack*, where an attacker derives keys for commonly used passwords, and tries to see if they work for different users' keychains. Salting slows the attackers down since they will have to do a separate dictionary attack for each user. To make this even more difficult, PBKDF2 is deliberately designed to be a slow function (it should take about 0.5-1s per evaluation on your laptop) so as to rate limit an attacker trying to brute force it.

3 SubtleCrypto

3.1 Using SubtleCrypto

The `SubtleCrypto` API is exposed to you in `password-manager.js` via a variable called `subtle`. For example, to hash a message with SHA-256, you could write in code:

```
let msgHash = await subtle.digest("SHA-256", stringToBuffer("CS 255 Rocks!"));
```

Note that `stringToBuffer` is a type conversion function that we provide. More details are below.

We encourage you to check out the docs to find out how to use the algorithms required for this project! You may generally use SHA-256 as a hash function when one is required.

3.2 Key Usages

When building a key using `SubtleCrypto`, you must specify the types of operations that the key may be used for. When building a key, there will be a `keyUsages` parameter, which should be filled with an array of strings specifying the different actions one might take with the key.

If the key will be used to generate other keys, it requires the `"deriveKey"` usage. For MACs, you will need to give the key the `"sign"` and `"verify"` usages. For ciphers, you will use `"encrypt"` and `"decrypt"`. It is generally good practice to only enable keys to be used for the purposes relevant to your use case!

3.3 Importing Keys in SubtleCrypto

In class, we consider keys to simply be arbitrary bitstrings. The `SubtleCrypto` library stores keys as a slightly larger object which includes metadata such as the algorithm and specific operations for

which the key may be used. Thus, in order to use user-defined data as a key, it must be imported via the `importKey` function.

```
let rawKey = await subtle.importKey("raw", stringToBuffer(password),
    "PBKDF2", false, ["deriveKey"]);
```

Note that `stringToBuffer` is a convenience function provided by the starter code which will be described in **??**.

The above snippet creates a new `CryptoKey` (the object representing keys in `SubtleCrypto`) whose contents are the raw unencrypted bytes of your specified password. In addition, this key may be only used for the PBKDF2 algorithm in order to derive a new key. **Note: This key is NOT sufficiently secure to use as the main key of your password manager. You must use PBKDF2 to produce a more cryptographically secure key.**

As another example, you will create your secret AES-GCM key as follows:

```
let keyMaterial = ... // get a random key by evaluating HMAC at an arbitrary value
let aesKey = await subtle.importKey(
    "raw", keyMaterial, "AES-GCM", true, ["encrypt", "decrypt"]);
```

Again, this produces a key from "raw" bytes, but these bytes are random because they are the result of HMAC. This key may be used for the AES-GCM algorithm to encrypt and decrypt data. See the API docs for more details.

4 API description

Here are descriptions of the functions you will need to implement. For each function, we also prescribe the run-time your solution must achieve (as a function of the number of entries n in the password database). We will assume that the input values (domain names and passwords) are of length $O(1)$, and regard each operation on an efficient dictionary/object/in-memory-key-value-store as a single step.

4.1 `static async init(password)`

- `password`: password used to protect the keychain (`string`)
- Returns: `Keychain` object
- Run-time: $O(1)$

This static method should create a new KVS. This function is also responsible for generating the necessary keys you need to provide for the various functionality of the password manager. Once initialized, the returned password manager should be in ready to support the other functionality described in the API.

4.2 `static async load(password, representation, trustedDataCheck)`

- `password`: password used to authenticate keychain (`string`)
- `repr`: JSON encoded serialization of the keychain (`string`)
- `trustedDataCheck`: SHA-256 hash of the keychain; note that this is an **optional** parameter that is used to check integrity of the password manager (`string`)
- Returns: `Keychain` object
- Run-time: $O(n)$

This static method loads the keychain state from a serialized representation. You can assume that `repr` is a valid serialization generated by a call to `keychain.dump()`. This function should verifythat the given `password` is valid for the keychain. If the parameter `trustedDataCheck` is provided,[4] this function should also affirm the integrity of the KVS. If tampering is detected, this function should throw an exception. If everything passes, the function should return the `Keychain` object represented by `repr`. if `password` is invalid, the function should throw an exception.

4.3 `constructor(...)`

- Your constructor may accept any parameters you would like.
- No return value
- Run-time: $O(1)$
- We will never invoke your constructor directly. Rather, we expect that your implementations of `init` and `load` will call it in some way.
- After the constructor has completed, we should be able to successfully call any other function in this API.

4.4 `async dump()`

- Returns: `array` consisting of two `string`'s, where the first is a JSON encoded serialization of the keychain and the second is a SHA-256 hash of the contents of the keychain
- Run-time: $O(n)$

This method should create a JSON encoded serialization of the keychain, such that it may be loaded back into memory via a subsequent call to `keychain.load`. It should return an array consisting of this, and also of the SHA-256 hash of the keychain contents (which is to be stored in trusted storage, and used to prevent rollback attacks).

[4] In Javascript, if an argument to a function is not provided, its value will be the special sentinel value `undefined`. You can test that a value is not `undefined` using the expression: `x !== undefined`.

4.5 `async set(name, value)`

- `name`: domain name of entry to add to the password manager (`string`)
- `value`: value associated with the given domain to store in the password manager (`string`)
- No return value
- Run-time: $O(1)$

This method should insert the domain and associated data into the KVS. If the domain is already in the password manager, this method will update its value. Otherwise, it will create a new entry.

4.6 `async get(name)`

- `name`: domain name of entry to fetch (`string`)
- Return: `string` (the value associated with the requested domain, `null` if not found)
- Run-time: $O(1)$

If the requested domain is in the KVS, then this method should return the the saved data associated with the domain. If the requested domain is not in the KVS, then this method should return `null`.

4.7 `async remove(name)`

- `name`: domain name of entry to fetch (`string`)
- Return: `boolean` (`true` if record with the specified name is found, `false` otherwise)
- Run-time: $O(1)$

If the requested domain is in the KVS, then this method should remove the record from the KVS. The method returns `true` in this case. Otherwise, if the specified domain is not present, return `false`.

5 Hints and Summary

5.1 Setup Instructions

You should run your code using Node.js. The setup is fairly straightforward: if you don't have it already, install Node on your system by downloading it from https://nodejs.org/en/download/. **If you already have Node installed, please make sure to update to the latest version since there may be issues with the autograder with older Node versions**. After installing it, you should be able to run the `node` and `npm` commands in your command line. Then, extract the starter code and `cd` into the directory `proj1`. We have provided a simple test suite, which you can run using the command `npm test` from this directory.

The full set of tests we are running to grade your assignment are provided on Gradescope. You can check your assignment against these tests by uploading your solution file. Recall that you will have to explain why your code is secure in the short-answer section.

5.2 Datatypes

In the past, a lot of confusion has been generated by the use of different JavaScript types throughout this application. The API that you will implement will take inputs (such as the master password, domain names, and website passwords) as regular strings because this is the most natural type for a client interacting with your library. The `SubtleCrypto` library uses buffer objects as inputs and outputs for all of its functions because these can contain arbitrary binary data. This includes auxiliary data such as salt, IV's, and associated data for authenticated encryption. These buffers can actually be a variety of different JavaScript types (`Buffer`, `ArrayBuffer`, `Uint8Array`) but these are all interchangeable for this project. The problem with these buffer types is that they do not work as keys in a JavaScript object and they do not serialize well using JSON.stringify. Thus we need a third representation for binary data that can be used in the KVS and dumped and loaded easily. We recommend using Base64 encoded strings, and we provide helper functions to convert back and forth between buffers and Base64 strings.

The provided conversion functions are as follows:

- `stringToBuffer(string)`

  Converts a plaintext string into a buffer that can be used with `SubtleCrypto` functions. Use this to pass a domain name into `subtle.sign`, for example.

- `bufferToString(buffer)`

  The inverse of `stringToBuffer`. Converts a buffer representing a plaintext (for example, the result of decryption) back into a string.

- `encodeBuffer(buffer)`

  Converts a buffer object containing arbitrary bytes into a Base64 encoded string. This string will work as the key in a JavaScript object and will be serialized correctly. It is important to use this function rather than trying to dump a buffer object directly.

  Note that Base64 strings may be compared for equality, but you should not attempt to modify the underlying binary data by manipulating a Base64 string.

- `decodeBuffer(base64)`

  The inverse of `encodeBuffer`. Used to convert Base64 strings back to buffers for use in `SubtleCrypto` functions.

The implementation of these functions can be found in the `lib.js` file of the starter code.

5.3 Implementation Details

- All the code you will have to write will be in the file `password-manager.js`. Please do **not** write any code in another file, since our auto-grader will be assuming that you haven't.

- `SubtleCrypto` is an inherently asynchronous library! If you are running into errors about values being undefined, check to make sure that each call to one of the functions in the library (using the imported `subtle` variable) is either preceeded by an `await` or followed bya `.then()`. For resources on asynchronous programming in Javascript, refer to Section 5.4.
- You can have a look at the tests being run in the file `test/test-password-manager.js`. You are always welcome to write more tests to make sure your implementation satisfies the requirements, but you are not required to, and we will not be grading your tests. The tests are written using the MochaJS framework (`https://mochajs.org/`) with Expect.js for assertions (`https://github.com/Automattic/expect.js`), and should be fairly readable.
- Serialization and deserialization of your password database (for the `load` and `dump`) functions should be done using JSON, via the standard Javascript APIs: `JSON.stringify` and `JSON.parse`. Further, the object that you serialize should have the key-value store that you use for your domain names and passwords stored internally in a field called `kvs` on the object (and you should use this field when you're restoring your keychain – in particular, if this field is modified, your object should have a modified set of domains and passwords). This is justto facilitate autograding. The last test in our suite is there to help you make sure that you're including this field (although it does *not* make sure you're using it when restoring).
- If your application detects tampering with any of its values at any point (like, say, a swap attack), it should throw an exception. We will not test what exception is thrown; it is fine to throw a string with an English description of the potential tampering.
- If you need random data, we provide a `getRandomBytes()` function in `lib.js`.
- All functions should make *at most* one call to PBKDF2.
- The *only* thing you should assume about SHA-256 is that it is collision resistant.

5.4 Summary of requirements

To summarize, you must implement a secure password manager that satisfies the following properties:

- The underlying in-memory data structure for the password manager should be a key-value store (Javascript object), where the keys correspond to domain names and the values correspond to the passwords for the given domain names.
- The password manager should be protected by a master password. Your implementation cannot include the master password (or any values that would leak information about it) in the serialized password database.
- When you need to derive a key from a password, you should use PBKDF2.
- Your system should satisfy the security properties described in the threat model (Section 2.2). In particular, you should defend against swap attacks and rollback attacks.

- You should implement all of the API functions (Section 4) with the parameters described there. Notably, your defenses for swap attacks and rollback attacks should be *independent*—your system must continue to be secure against swap attacks even if a hash value from trusted storage is not provided.
- You also need to submit answers to the short-answer questions from Section 6.

Please submit only your password-manager.js file to the "Project 1 Code Submission" assignment when you are finished.

5.5 Helpful Resources

- A guide to async/await in Javascript *here*.
- A guide to Promises in Javascript *here*.
- The SubtleCrypto docs *here*.

6 Short-answer Questions

In addition to your implementation, please include answers to the following questions regarding your implementation. Your answers need not be long, but should include important details.

Please submit (preferably) typed or handwritten answers to the "Project #1 Short-answer Questions" assignment (separate from programming compo- nent).

1. Briefly describe your method for preventing the adversary from learning information about the lengths of the passwords stored in your password manager.
2. Briefly describe your method for preventing swap attacks (Section 2.2). Provide an argument for why the attack is prevented in your scheme.
3. In our proposed defense against the rollback attack (Section 2.2), we assume that we can store the SHA-256 hash in a trusted location beyond the reach of an adversary. Is it necessary to assume that such a trusted location exists, in order to defend against rollback attacks? Briefly justify your answer.
4. Because HMAC is a deterministic MAC (that is, its output is the same if it is run multiple times with the same input), we were able to look up domain names using their HMAC values. There are also randomized MACs, which can output different tags on multiple runs with the same input. Explain how you would do the look up if you had to use a randomized MAC instead of HMAC. Is there a performance penalty involved, and if so, what?
5. In our specification, we leak the number of records in the password manager. Describe an approach to reduce the information leaked about the number of records. Specifically, if there are k records, your scheme should only leak $\log_2(k)$ (that is, if k_1 and k_2 are such that $\log_2(k_1) = \log_2(k_2)$, the attacker should *not* be able to distinguish between a case where the true number of records is k_1 and another case where the true number of records is k_2).

6. What is a way we can add multi-user support for specific sites to our password manager system without compromising security for other sites that these users may wish to store passwords of? That is, if Alice and Bob wish to access one stored password (say for nytimes) that either of them can get and update, without allowing the other to access their passwords for other websites.